

Aprendizaje profundo (*Deep Learning*)

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- La última frontera del curso: el **aprendizaje profundo**
 1. Redes neuronales artificiales (ANNs)
 2. Redes convolucionales (CNNs) — imágenes
 3. Redes recurrentes (RNNs) — secuencias
- Las ANNs las haremos en R (`neuralnet`); las CNN/RNN las veremos con **keras** (para correr en Google Colab)

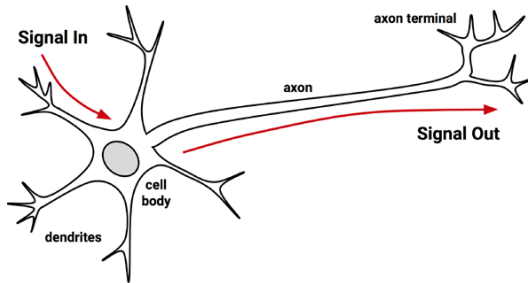
Parte 1

Redes neuronales artificiales

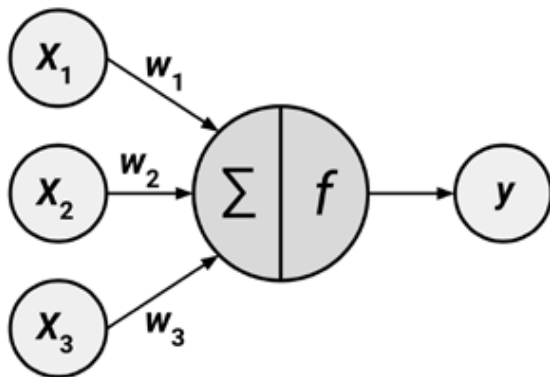
Inspiración: el cerebro

- El cerebro es una red de **neuronas** interconectadas, un procesador paralelo masivo
- Cada neurona capta señales, las procesa y, si superan un **umbral**, emite una señal
- Las **redes neuronales artificiales** imitan esa relación entre señales de entrada y de salida

Neuronas biológicas



Neuronas artificiales



Neuronas artificiales

Una neurona con n entradas se representa como:

$$y(x) = f\left(\sum_{i=1}^n w_i x_i\right)$$

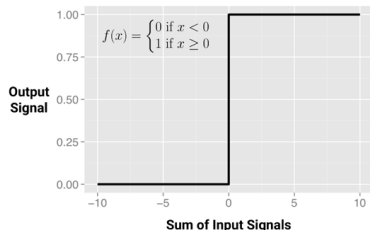
- Cada entrada x_i se pondera por un **peso** w_i (su importancia)
- La **función de activación** $f(\cdot)$ decide la señal de salida a partir de la suma ponderada

Tres componentes de una red

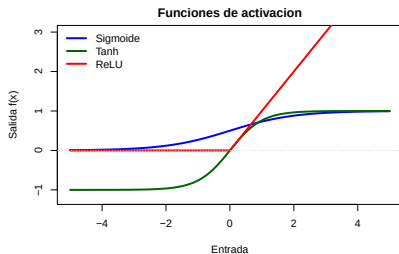
1. **Función de activación:** transforma la suma de entradas en una salida
2. **Topología:** número de neuronas, de capas, y cómo se conectan
3. **Algoritmo de entrenamiento:** cómo se fijan los pesos de las conexiones

Funciones de activación

- La biológica es casi dicotómica: se cruza el umbral o no

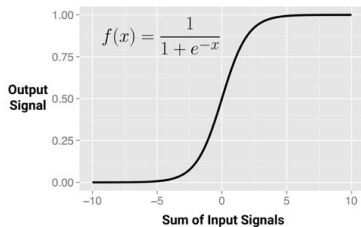


- Las artificiales son **suaves** y diferenciables (útil para optimizar): sigmoide, tanh, ReLU



Función sigmoideal

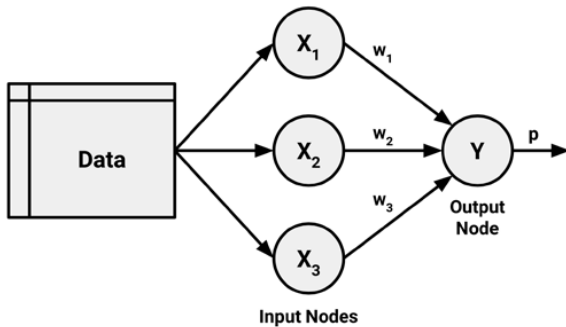
- Salida continua entre 0 y 1
- Diferenciable (clave para entrenar)
- Muy popular; hoy ReLU domina en redes profundas



Topología: capas

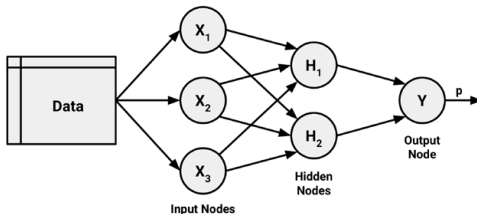
- Las neuronas se organizan en **capas**: una de **entrada**, una de **salida**, y posibles capas **ocultas**
- Cada nodo de entrada procesa una característica; el de salida produce la predicción
- La versión más simple tiene una sola capa

Red de una sola capa



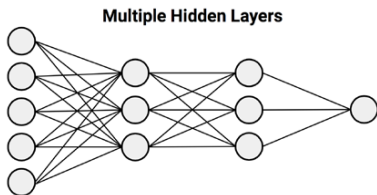
Redes multicapa

- Las capas **ocultas** procesan las señales antes de la salida; suelen estar totalmente conectadas



Deep learning

- Una red con **muchas capas ocultas** es una **red neuronal profunda**
- Entrenarla es *deep learning*
- Brilla con datos **no estructurados**: imágenes, sonido, texto



¿Cuántos nodos y capas?

- Los nodos de **entrada** los fija el número de características; los de **salida**, el problema
- Los nodos **ocultos** los elige el usuario: no hay regla teórica
- Más capacidad → puede aprender relaciones más complejas...
- ...pero mayor riesgo de **sobreajuste** (de nuevo, el sesgo-varianza)

Entrenamiento: backpropagation

- Se parte de pesos **aleatorios** y se itera en ciclos (*épocas*), cada uno con dos fases:
 1. **Hacia adelante**: se activan las neuronas de la entrada a la salida y se produce una predicción
 2. **Hacia atrás**: se compara con el valor real y el **error** se propaga para ajustar los pesos
- El ajuste usa **descenso de gradiente**: mover los pesos en la dirección que más reduce el error

Al taller

- Predeciremos el **gasto médico anual** de un asegurado a partir de sus características (economía de la salud)
- Base `insurance.csv`, con el paquete `neuralnet` (el mismo problema que vimos con regresión en S3)
- Compararemos una red con 1 nodo oculto contra una con 5
- (Este taller **sí** corre en R sin configuración adicional)

Parte 2

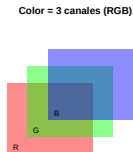
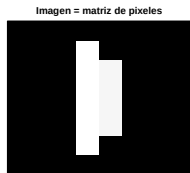
Redes convolucionales (CNNs)

¿Por qué redes especiales para imágenes?

- Una imagen pequeña (100×100) ya tiene 10.000 píxeles
- Una red **densa** necesitaría millones de pesos: no escala y sobreajusta
- Además, ignoraría la **estructura espacial**: píxeles vecinos están relacionados
- Las **CNN** aprovechan esa estructura local

Una imagen es una matriz de números

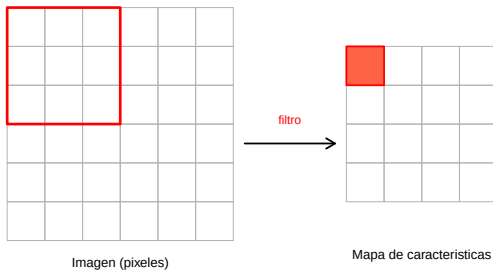
- Cada **píxel** es un número (su intensidad)
- A color son **3 matrices** (canales rojo, verde, azul)
- Una imagen = un **tensor** alto $\times$ ancho $\times$ canales



La convolución

- Un **filtro** (pequeña matriz de pesos) recorre la imagen y detecta un patrón local (un borde, una textura)

Convolucion: un filtro recorre la imagen



La convolución, en detalle

- En cada posición, el filtro multiplica **punto a punto** con la ventana y **suma**: un solo número

Convolucion: producto punto + suma

1	0	1
0	1	1
0	0	1

ventana (pixeles)

 $*$

1	0	1
0	1	0
1	0	1

filtro (pesos)

 $=$

4

un valor del
mapa de
características

- Al deslizarse por toda la imagen, produce un **mapa de características**

Filtros y parámetros compartidos

- Cada filtro produce un **mapa de características**: dónde aparece ese patrón
- Los pesos del filtro **se aprenden** (no se diseñan a mano)
- El **mismo** filtro se usa en toda la imagen: **comparte parámetros**
 - ▶ muchísimos menos pesos que una red densa
 - ▶ detecta el patrón **esté donde esté** (invarianza a traslación)

Una jerarquía de características

Las capas aprenden una jerarquía

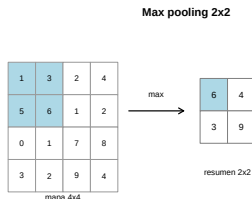


capas tempranas --> capas profundas (mas abstracto)

- Al apilar capas, la red aprende sola: de **bordes** a **partes** a **objetos** completos

Pooling

- Resume regiones (p. ej. el **máximo** de cada vecindad 2×2)
- Reduce el tamaño (menos cómputo)
- Da **invarianza**: el patrón se detecta aunque se mueva un poco



Ingredientes que faltan

- **ReLU** como activación: rápida y evita que los gradientes se desvanezcan
- **Softmax** en la última capa: convierte la salida en probabilidades por clase
- **Dropout**: apagar neuronas al azar durante el entrenamiento, para **regularizar** y combatir el sobreajuste

Arquitecturas famosas y *transfer learning*

- Hitos: LeNet (dígitos), **AlexNet** (2012, el “big bang” del deep learning), VGG, ResNet
- **Transfer learning**: tomar una red ya entrenada en millones de imágenes y **reajustarla** a tu problema
- Permite obtener buenos resultados con **pocos datos** propios: hoy es la práctica habitual

Aplicaciones

- Reconocimiento de objetos y rostros
- Diagnóstico por imágenes médicas (rayos X, resonancias)
- Carros autónomos, imágenes satelitales (deforestación, cultivos)
- **AI taller:** clasificar dígitos escritos a mano con la base **MNIST** (en keras)

Parte 3

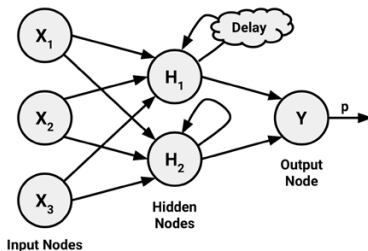
Redes recurrentes (RNNs)

Datos secuenciales

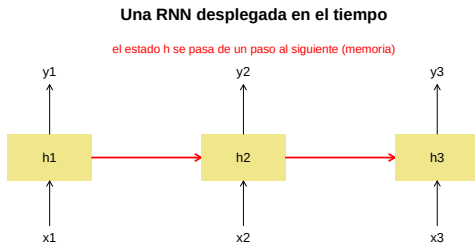
- Muchos datos son **secuencias** donde el **orden** importa:
 - ▶ texto (una palabra depende de las anteriores)
 - ▶ series de tiempo (precios, clima)
 - ▶ audio, video
- Una red normal trata cada entrada por separado: no tiene **memoria** del pasado

La idea: recurrencia y memoria

- Una **RNN** tiene conexiones que vuelven hacia atrás (*loops*)
- Mantiene un **estado** que resume lo visto hasta ahora
- Así incorpora **memoria** de corto plazo



Una RNN desplegada en el tiempo



- En cada paso t entra x_t , se actualiza el estado h_t (usando h_{t-1}) y se produce y_t

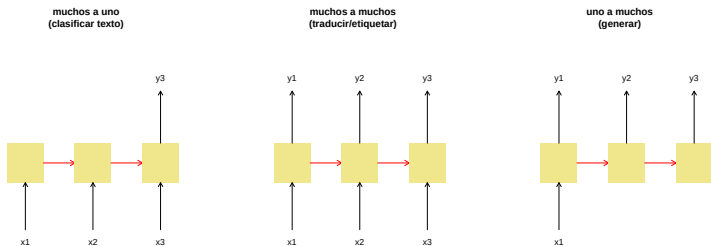
¿Cómo procesa una RNN?

- En cada paso combina la entrada x_t con el estado previo h_{t-1} :

$$h_t = f(W x_t + U h_{t-1} + b)$$

- El estado h_t **resume** todo lo visto hasta el momento (la memoria)
- Los **mismos** pesos (W , U) se reutilizan en cada paso: comparte parámetros **en el tiempo** (como el filtro de la CNN lo hace en el espacio)

Tipos de tareas secuenciales



- La misma idea sirve para clasificar, traducir/etiquetar o generar secuencias

El gradiente que se desvanece

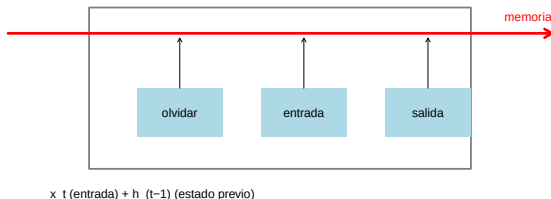
- Para entrenar, el error se propaga muchos pasos **hacia atrás en el tiempo**
- Al multiplicarse paso tras paso, los gradientes tienden a **encogerse** (o explotar)
- Resultado: la red “olvida” lo lejano y le cuesta aprender **dependencias largas**
- Es el problema que motivó a las LSTM

LSTM y GRU

- Las **LSTM** añaden una **celda de memoria** con “compuertas” que deciden qué recordar y qué olvidar
- Así manejan mucho mejor las dependencias de **largo plazo**
- Las **GRU** son una versión más simple, con menos compuertas
- Son el estándar cuando se usan RNN en la práctica

LSTM: las compuertas

Celda LSTM: compuertas que regulan la memoria



- **Olvidar** (qué borrar), **entrada** (qué guardar), **salida** (qué emitir): regulan el flujo de la memoria

Atención

- En vez de comprimir todo en un estado, la **atención** deja que cada elemento “mire” directamente a los relevantes
- Cada palabra **pondera** cuánto atiende a las demás



Transformers y modelos de lenguaje

- Los **Transformers** reemplazan la recurrencia por pura **atención**
- Procesan toda la secuencia **en paralelo**: más rápido y mejor con dependencias largas
- Son la base de los grandes **modelos de lenguaje** (la “T” de ChatGPT)
- Su punto de entrada son los **embeddings** (sesión 5): el texto convertido en vectores

Aplicaciones y al taller

- Traducción automática, generación de texto, reconocimiento de voz
- Pronóstico de series de tiempo
- **AI taller**: clasificar reseñas de cine como positivas/negativas con la base **IMDB**, usando una capa de *embedding* + una **LSTM** (en keras)

¿Cuándo (y cuándo no) usar deep learning?

- Brilla con **muchos datos** y entradas **no estructuradas** (imágenes, texto, audio, video)
- Para datos **tabulares** y muestras **pequeñas**, métodos como regresión, árboles o *boosting* (XGBoost) suelen igualar o superar al DL, y son más rápidos e interpretables
- Costos del DL: mucho **cómputo**, muchos **datos**, y es una **caja negra**
- Regla práctica: empieza simple y usa DL cuando el problema y los datos lo justifiquen

Herramientas y cómputo

- *Frameworks* más usados hoy: **TensorFlow/Keras** y **PyTorch**
- El entrenamiento se hace en **GPUs** (mucho más rápido que en CPU)
- Por eso, para los talleres de CNN/RNN usaremos **Google Colab**, que ofrece GPU gratis y ya trae todo instalado
- Y aprovecharemos modelos **preentrenados** (*transfer learning*) cuando se pueda

Resumen de la sesión

- Las **ANNs** imitan neuronas: suma ponderada + activación, en capas, entrenadas con *backpropagation*
- **CNNs**: convolución (parámetros compartidos) + pooling para **imágenes**; jerarquía de características
- **RNNs** (y LSTM): memoria para **secuencias**; la **atención** y los **Transformers** son el presente del área

Cierre del curso

En seis sesiones recorrimos:

- I. Clasificación: KNN, Naive Bayes, SVM, árboles
- II. Predicción numérica: regresión, Ridge, Lasso
- III. Desempeño: evaluación, variables, desbalance, ensamblaje
- IV. No supervisado: clustering, anomalías, representación
- V. Aprendizaje profundo: ANNs, CNNs, RNNs

¡Gracias!